How the Recoverability Analysis Works

The analysis is the research; SynapseOS is its platform. There, a local language model turns a request into a shell command, and the analysis checks it before it runs: what it will change, and whether it can be undone. **Only the translation uses AI. The analysis itself does not use the language model**: it is deterministic program analysis that checks the model's output, specified in Chapter 3, §2.1(a)–(c) (pp. 64–70).

Where the analysis sits

INPUT
User request
"Clean up my old screenshots."

→

LANGUAGE MODEL
Translates
Writes a shell command. qwen2.5-coder:3b, run locally.

→

RECOVERABILITY ANALYSIS
Checks
Derives the command's effects from its structure. No machine learning.

→

SAFETY GATE
Acts
Backs up the targets, or asks first when they cannot be determined.

→

UNDO JOURNAL
Restores
Reverses the change from the backups.

Probabilistic: the same request may yield different commands. **Deterministic:** the same command always yields the same decision.

Inside the analysis, in five steps

Running example: `find . -name "Screenshot*.png" -delete`

1	**Parse** Reads the command's structure with a shell-language parser.	Program `find`; search path `.`; filter `-name "Screenshot*.png"`; action `-delete`.
2	**Look up** A rule table records what each program does to files.	`find … -delete` removes every file that matches its filter; which files those are is not named in the command.
3	**Compose** Sees through searches, pipes, loops and wrappers to the program that acts.	The deletion is carried out by `find` itself on files it discovers while running. The same reasoning applies to `find -exec rm`, `| xargs rm`, `for` loops and `sh -c`.

4	**Resolve**
	Runs only the proven read-only part, in a sandbox where nothing can be written.
		ILLUSTRATION (SIMPLIFIED) OF AN INTERNAL STEP; USERS NEVER SEE IT:
		`$ find . -name "Screenshot*.png"   # -delete replaced by a read-only print`
		`./Screenshot 2026-09-20 184502.png`
		`./Screenshot 2026-09-14 101211.png`

5	**Decide**
	Classifies the effects and plans what to back up.
		ACTUAL ANALYSIS OUTPUT, UNEDITED:
		`command:    find . -name "Screenshot*.png" -delete`
		`composition: enabled (full algorithm)`
		`class:      recoverable-with-capture`
		`  remove   ./Screenshot 2026-09-20 184502.png`
		`  remove   ./Screenshot 2026-09-14 101211.png`
		`plan:       what is captured before it runs`
		`  trash    ./Screenshot 2026-09-14 101211.png`
		`  trash    ./Screenshot 2026-09-20 184502.png`
		Both files are backed up before the command runs: *trash* keeps a hard link to each, so the backup takes no extra space. Had their effects been undeterminable (an unknown program, inline Python, `shred`), the command would be classified *unrecoverable* and confirmation required.

Why it matters

It protects data, not intent

The decision is made by code, not a model, so a wrong command that would destroy files is still caught. It cannot tell whether a command is what you meant: a wrong but harmless one runs. That part is only as good as the model.

What it improves on

Coding agents check commands against a list of dangerous patterns and rely on git to undo mistakes. Personal files have no git, and a list misses targets hidden in `find` or `xargs`. On 777 test commands (§3.1(d)), a list let 41% of destructive ones run unprotected; this analysis let none, and backed up 89.5%.

It is the computer-science contribution

The analysis is static program analysis in the tradition of abstract interpretation (p. 17), not machine learning. Without composition and resolution, backups drop from 89.5% to 32% (§3.2).

Inside the Algorithm

The algorithm in plain language, and where each part lives in the code. Its four parts (EFFECTS, RESOLVE, VERDICT, PLAN) are the four functions of Algorithm 1 in Chapter 3 (p. 68), which states them formally. Everything here is deterministic program analysis; none of it calls the language model.

The algorithm, step by step

Start here — given a command and the folder it will run in

1. Break the command into its parts, the way a sentence breaks into clauses.
2. Keep a notepad of files that earlier parts will create or delete, so later parts see the folder as it will be by then.
3. Work out what every part does to files **→ EFFECTS** (which calls **RESOLVE** when a command's files are only known while it runs).
4. Decide whether the user must be asked first. **→ VERDICT**
5. If a backup makes it undoable, choose the cheapest backup that is enough. **→ PLAN**

EFFECTS — what does this part do to files?

- **One command**, like `rm a.txt`: look it up in the rule book.
- **Commands joined together**, like `a && b` or `a | b`: work out both; the second sees the first's changes on the notepad.
- **Output sent to a file**, like `> notes.txt`: that file gets written.
- **A loop** over several items: work out its body once for each item.
- **A wrapper**, like `sudo` or `sh -c`: work out the command inside it.
- **Files known only while it runs**, like `find … -delete` or `xargs`: find those files first, safely **→ RESOLVE**. If they can't be found, mark the command "unknown"; otherwise work out the action once per file.

RESOLVE — find those files without changing anything

- Make a harmless version of the command, e.g. a `find` that only lists matches instead of deleting them.
- Run it only if it is proven unable to change anything; otherwise give up, and the command stays "unknown".
- The files it lists are the command's real targets.

VERDICT — ask first, or not?

- Anything "unknown", or something no backup can undo (e.g. `shred`) → **ask first** `UNRECOVERABLE`
- It only creates or moves files → **just run** `RECOVERABLE`
- Otherwise → **back it up, then run** `RECOVERABLE_WITH_CAPTURE`

PLAN — the cheapest backup that is enough

- Deleted file → a hard link: a second name for it, costing no space.
- Overwritten file → a copy. Changed permissions → a note of the old ones.
- Skip files inside a folder that is already backed up.

File map: `prototype/internal/`

```
prototype/
└── internal/
    ├── effects/          the algorithm
    │   ├── types.go      effects, issues, the result
    │   ├── analyze.go    walk, notepad  (EFFECTS)
    │   ├── wrappers.go   sudo, sh -c, find, xargs
    │   ├── rules.go      per-command rules  (RULE)
    │   ├── adapters.go   dry runs  (RESOLVE)
    │   ├── runner.go     read-only sandbox
    │   ├── pkgstate.go   packages, services
    │   ├── verdict.go    VERDICT, PLAN
    │   ├── failclosed.go  baseline: fail-closed list
    │   ├── catalog.go    rule notes for the docs
    │   └── *_test.go     6 files, 53 tests, fuzz
    ├── gate/             uses it  ask, or back up
    ├── undo/             uses it  journal, restore
    ├── classifier/       evaluates it  pattern list
    ├── baselines/        evaluates it  Codex CLI
    ├── oracle/           evaluates it  ground truth
    ├── recoverycheck/    evaluates it  undo check
    └── ollama/ toolcall/ executor/ session/
        loopevent/ telemetry/ tui/ typedops/
                          rest of SynapseOS
```

Guarantees, and how they are checked

- **Fails safe.** Anything it cannot work out is marked "unknown", and unknown always means asking first.
- **Changes nothing while analysing.** A fuzz test fed 336,639 random command lines; every command the analysis ran was read-only.
- **Deterministic.** The same command always gets the same answer; no AI in this code.
- **Measured.** 53 test functions, 85% of the analysis covered, enforced by `make ci`; evaluated on 777 commands (Chapter 3, §3.1).

A request through the code: the model writes the command → `gate.Decide` → `effects.Analyze` → **VERDICT**, **PLAN** → ask, or `gate.Capture` → run → `synapse undo`

Full reference: `docs/recoverability-analysis.md`